Préparation

```
import pandas as pd
import json, pickle, os
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
from sklearn.cluster import DBSCAN
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report
from xgboost import XGBClassifier
from sklearn.metrics import classification_report, confusion_matrix, silhouette_score, ConfusionMatrixDisplay
from pathlib import Path
import ruptures as rpt
import numpy as np
```

```
from pathlib import Path
```

```
COLORS = {'primary':'#E91E8C','secondary':'#7B2FBE','accent':'#00D4AA','dark':'#1A1A2E'}
PALETTE= ['#E91E8C','#7B2FBE','#00D4AA','#FF6B35','#4ECDC4','#45B7D1','#96CEB4','#FFEAA7']
plt.rcParams.update({'figure.facecolor':'white','axes.facecolor':'#FAFAFA',
                     'axes.grid':True,'grid.alpha':0.25,'font.size':11,
                     'axes.spines.top':False,'axes.spines.right':False})
report_models = {}

BASE      = Path('..')
DATA_PROC = BASE / 'data' / 'processed'
MODELS    = BASE / 'models'
for p in [DATA_PROC, MODELS, BASE/'data'/'raw']: p.mkdir(parents=True, exist_ok=True)
```

CLASSIFICATION

```
df = pd.read_csv("../data/processed/gdelt_republique_benin_clean.csv")

features = [
    'GoldsteinNorm',
    'ToneNorm',
    'NumMentions',
    'NumArticles',
    'MediaWeight'
]

X = df[features].dropna()

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

PCA

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

df_pca = pd.DataFrame(X_pca, columns=['PC1', 'PC2'])

print(pca.explained_variance_ratio_)
```

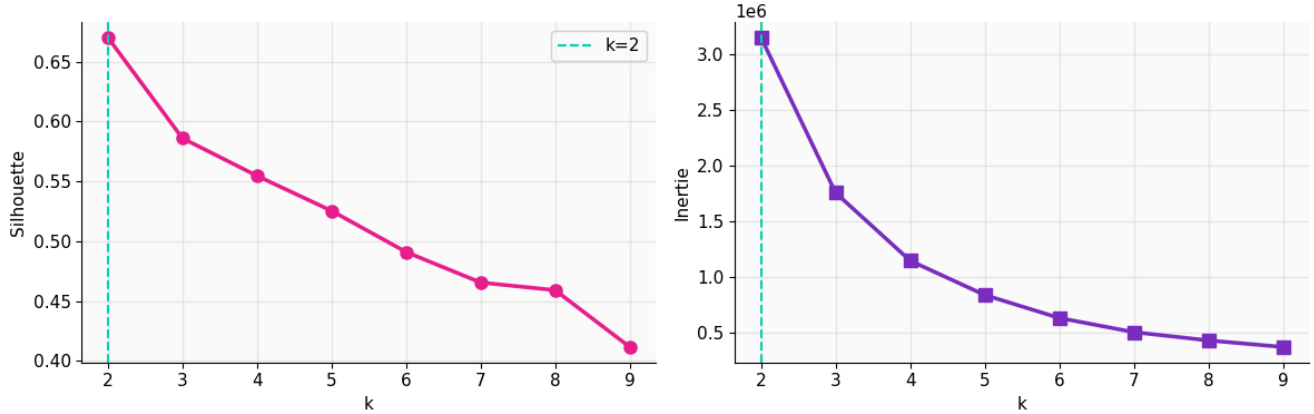
```
[0.38151723 0.30249973]
```

K-Means

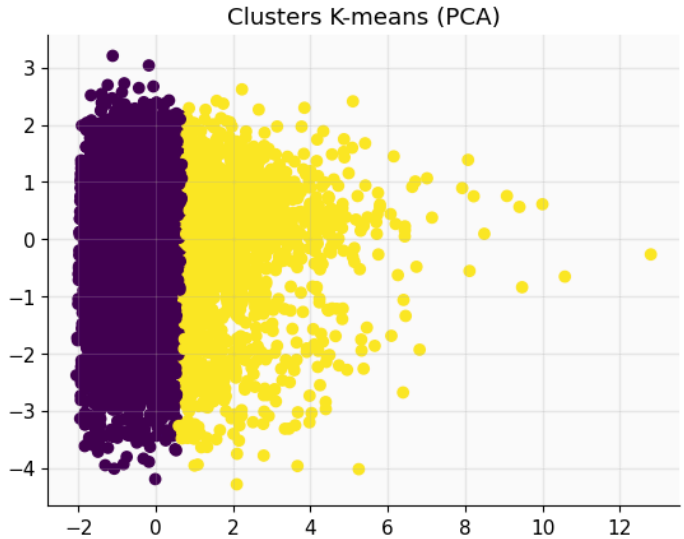
```
# K-Means optimal
sil,inert={},{}
```

```
for k in range(2,10):
    km=KMeans(n_clusters=k,random_state=42,n_init=10)
    lbl=km.fit_predict(X)
    sil[k]=silhouette_score(X,lbl,sample_size=2000)
    inert[k]=km.inertia_
    print(f'k={k}: silhouette={sil[k]:.4f}')
best_k=max(sil,key=sil.get)
print(f'\n✅ k optimal = {best_k}')

fig,axes=plt.subplots(1,2,figsize=(12,4))
axes[0].plot(list(sil.keys()),list(sil.values()),'o-',color=COLORS['primary'],lw=2.5,ms=8)
axes[0].axvline(best_k,ls='--',color=COLORS['accent'],label=f'k={best_k}')
axes[0].set_xlabel('k'); axes[0].set_ylabel('Silhouette'); axes[0].legend()
axes[1].plot(list(inert.keys()),list(inert.values()),'s-',color=COLORS['secondary'],lw=2.5,ms=8)
axes[1].axvline(best_k,ls='--',color=COLORS['accent'])
axes[1].set_xlabel('k'); axes[1].set_ylabel('Inertie')
plt.tight_layout(); plt.show()
```

```
k=2: silhouette=0.6701
k=3: silhouette=0.5861
k=4: silhouette=0.5547
k=5: silhouette=0.5254
k=6: silhouette=0.4910
k=7: silhouette=0.4655
k=8: silhouette=0.4591
k=9: silhouette=0.4116

✅ k optimal = 2
```



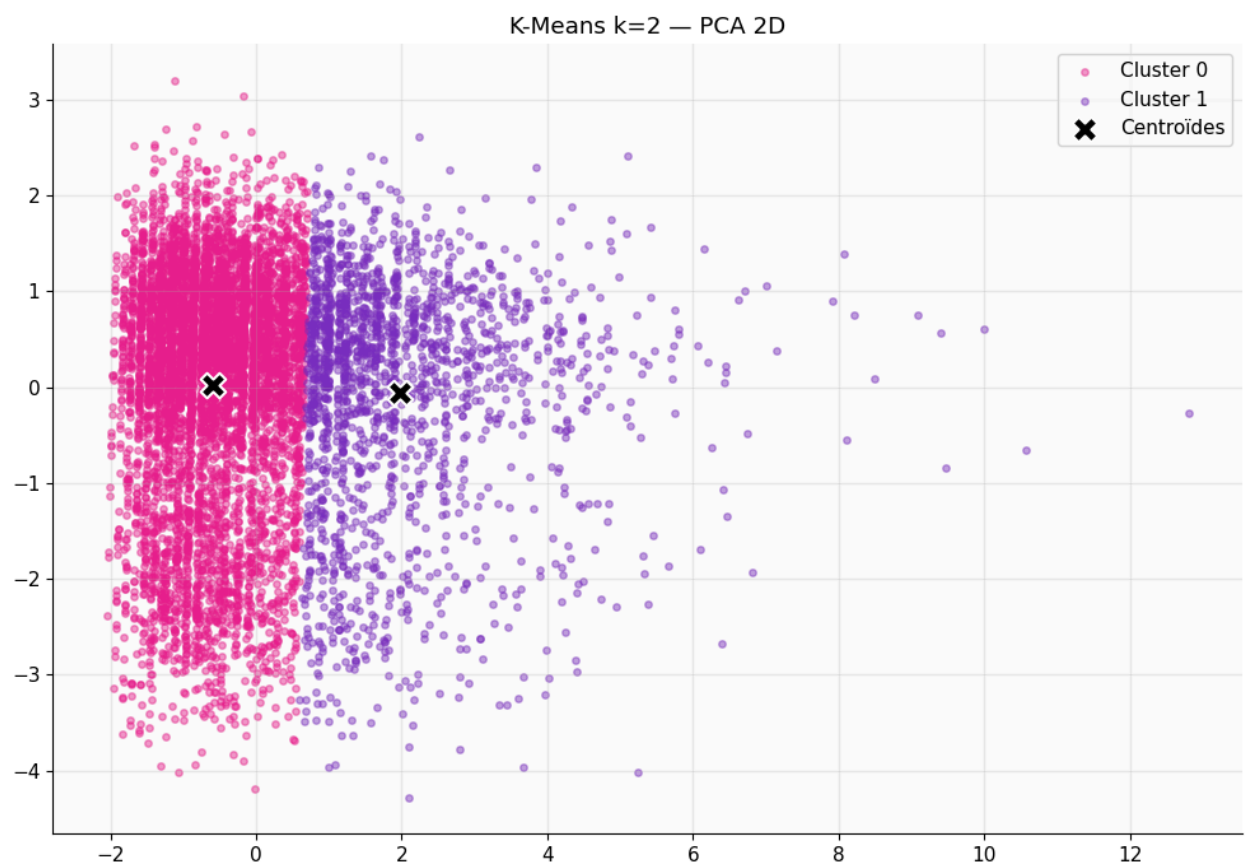

```
kmeans = KMeans(n_clusters=best_k, random_state=42)
clusters = kmeans.fit_predict(X_scaled)

df['cluster_kmeans'] = clusters

plt.scatter(X_pca[:,0], X_pca[:,1], c=clusters)
plt.title("Clusters K-means (PCA)")
plt.show()
```



```
df.columns
```

```
Index(['SQLDATE', 'date', 'Actor1Name', 'Actor1CountryCode', 'Actor2Name',
       'Actor2CountryCode', 'EventCode', 'EventLabel', 'QuadClass',
       'GoldsteinScale', 'NumMentions', 'NumArticles', 'AvgTone',
       'ActionGeo_FullName', 'ActionGeo_CountryCode', 'ActionGeo_Lat',
       'ActionGeo_Long', 'SourceDomain', 'GKG_Themes', 'GKG_NumThemes',
       'GKG_Persons', 'GKG_NumPersons', 'GKG_Organizations', 'GKG_NumOrgs',
       'GKG_V2Tone_Positive', 'GKG_V2Tone_Negative', 'GKG_V2Tone_Polarity',
       'GKG_V2Tone_ActivityDensity', 'GKG_V2Tone_SelfGroupDensity',
       'GKG_V2Tone_WordCount', 'GCAM_Anger', 'GCAM_Fear', 'GCAM_Joy',
       'GCAM_Sadness', 'GCAM_Trust', 'GCAM_Surprise', 'GCAM_DominantEmotion',
       'GKG_DocumentIdentifier', 'GKG_HasImage', 'GKG_Language',
       'GKG_IsTranslated', 'QuadLabel', 'IsConflict', 'IsNegative',
       'MediaWeight', 'ZoneBenin', 'DepartementBenin', 'PopulationVille',
       'IsNorthBenin', 'SourceType', 'GoldsteinNorm', 'ToneNorm', 'month',
       'GKG_ThemeConflict', 'GKG_ThemeEconomy', 'GKG_ThemeGovern',
       'GKG_ThemeHumanDev', 'GKG_ThemeEnviro', 'GKG_ToneGap',
       'GKG_EmotionIntensity', 'ToneCategory', 'GoldsteinBin',
       'cluster_kmeans'],
      dtype='object')
```

```
df['PCA1'],df['PCA2']=X_pca[:,0],X_pca[:,1]

fig,ax=plt.subplots(figsize=(10,7))
for i,c in enumerate(sorted(df['cluster_kmeans'].unique())):
    mask=df['cluster_kmeans']==c
    ax.scatter(df.loc[mask,'PCA1'],df.loc[mask,'PCA2'],s=15,alpha=.45,color=PALETTE[i],label=f'Cluster {c}')
centers_2d=pca.transform(kmeans.cluster_centers_)
ax.scatter(centers_2d[:,0],centers_2d[:,1],s=200,marker='X',color='black',edgecolors='white',lw=1.5,zorder=5,label='Centroïdes')
ax.set_title(f'K-Means k={2} — PCA 2D'); ax.legend()
plt.tight_layout()
# plt.savefig(DATA_PROC/'viz_clusters.png',dpi=150,bbox_inches='tight')
plt.show()

profile=df.groupby('cluster_kmeans').agg(
    Événements=('EventCode','count'),Tone_moyen=('AvgTone','mean'),
    Goldstein_moy=('GoldsteinScale','mean'),Thème_top=('EventLabel',lambda x:x.mode()[0]),
    Ville_top=('ActionGeo_FullName',lambda x:x.mode()[0]),
    Conflits_pct=('IsConflict',lambda x:f'{x.mean()*100:.0f}%')).round(2)
print(profile.to_string())
```




```
                Événements  Tone_moyen  Goldstein_moy             Thème_top      Ville_top Conflits_pct
cluster_kmeans
0                     6146        0.45           0.76  Declaration publique  Abomey-Calavi          23%
1                     1854        0.41           0.81  Declaration publique        Cotonou          23%
```

DBSCAN

```
dbscan = DBSCAN(eps=0.5, min_samples=5)
db_clusters = dbscan.fit_predict(X_scaled)

df['cluster_dbscan'] = db_clusters
```

```
df.groupby('cluster_kmeans')[features].mean()
```

cluster_kmeans	GoldsteinNorm	ToneNorm	NumMentions	NumArticles	MediaWeight
0	0.538024	0.511153	6.586398	3.193622	18.736251
1	0.540333	0.510125	12.493528	6.860841	76.339806

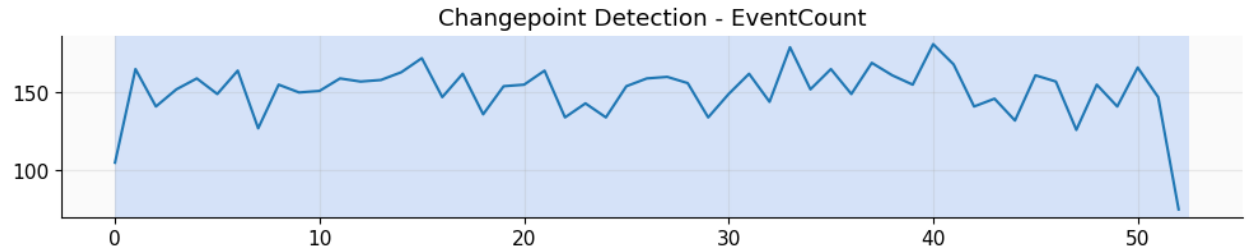
PERL

```
df['date'] = pd.to_datetime(df['date'], errors='coerce')
```

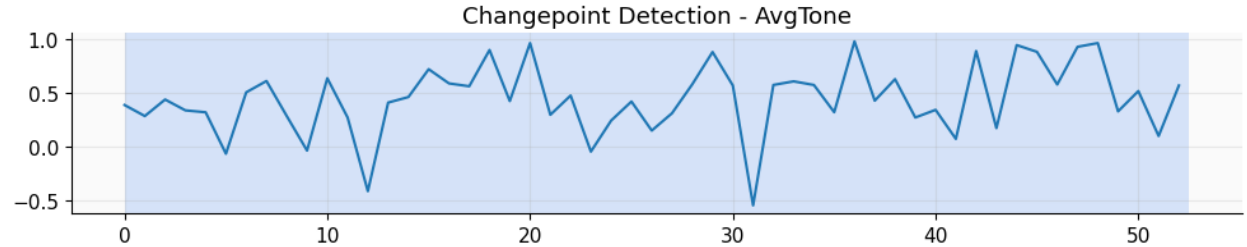
```
# VIZ 1 — Timeline
df_wk = df.groupby(df['date'].dt.to_period('W')).agg(EventCount=('AvgTone','count'), Golde_moyen=('GoldsteinScale','mean'), AvgTone=('
```

```
model_perl = {}
for col in ['EventCount', 'AvgTone', 'Golde_moyen']:
    signal = df_wk[col].values

    model = rpt.Pelt(model="rbf").fit(signal)
    bkps = model.predict(pen=2)
    model_perl[col] = model

    print(f"Rupture de {col} détectée aux semaines: ", bkps)
    rpt.display(signal, bkps)
    plt.title(f"Changepoint Detection - {col}")
    plt.show()
```

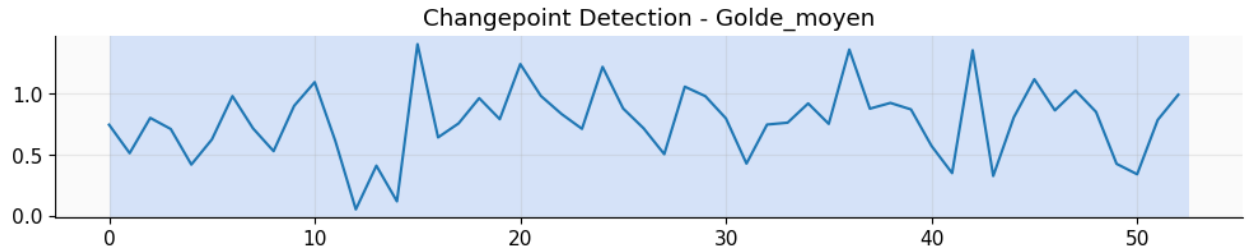
```
Rupture de EventCount détectée aux semaines:  [53]
```




```
Rupture de AvgTone détectée aux semaines:  [53]
```




```
Rupture de Golde_moyen détectée aux semaines:  [53]
```



PREDICTION

```
y = df['IsConflict']
```

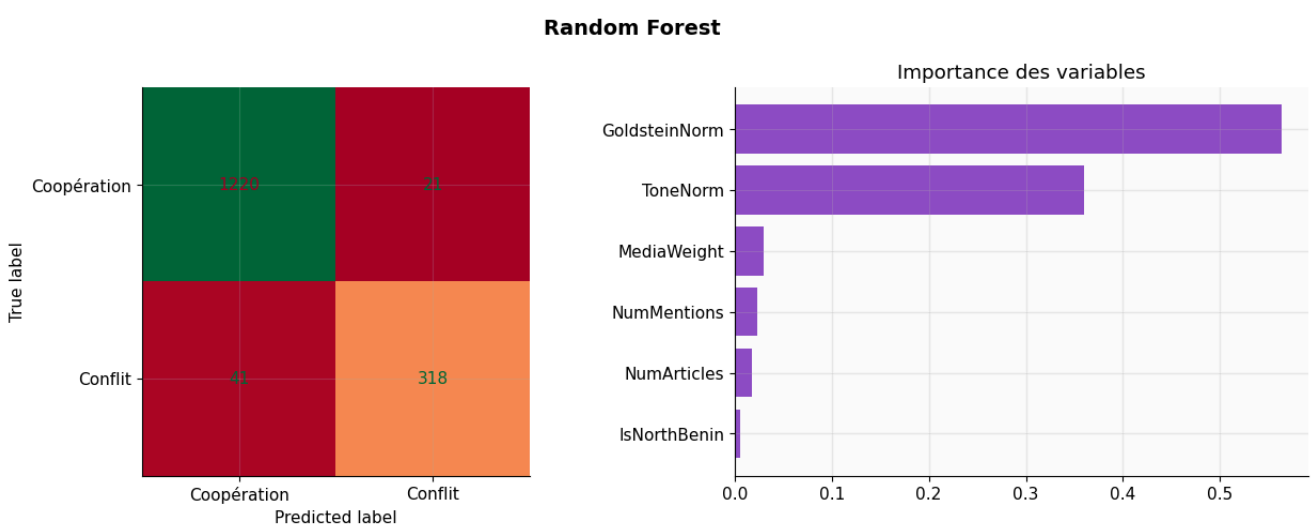
```
features = [
    'GoldsteinNorm',
    'ToneNorm',
    'NumMentions',
    'NumArticles',
    'MediaWeight',
    'IsNorthBenin'
]

X = df[features].fillna(0)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=123)
```

RANDOM FOREST

```
model_rf = RandomForestClassifier(n_estimators=100, random_state=42)
model_rf.fit(X_train, y_train)

y_pred = model_rf.predict(X_test)

# print(classification_report(y_test, y_pred))


report=classification_report(y_test,y_pred,target_names=['Coopération','Conflit'],output_dict=True)
report_models['rf'] = report
cm=confusion_matrix(y_test,y_pred)
print(classification_report(y_test,y_pred,target_names=['Coopération','Conflit']))

fig,axes=plt.subplots(1,2,figsize=(13,5))
fig.suptitle('Random Forest',fontsize=14,fontweight='bold')
ConfusionMatrixDisplay(cm,display_labels=['Coopération','Conflit']).plot(ax=axes[0],cmap='RdYlGn',colorbar=False)
imp=pd.Series(model_rf.feature_importances_,index=features).sort_values()
axes[1].barh(imp.index,imp.values,color=COLORS['secondary'],alpha=.85)
axes[1].set_title('Importance des variables')
plt.tight_layout()
plt.savefig(DATA_PROC/'viz_rf.png',dpi=150,bbox_inches='tight')
plt.show()
```

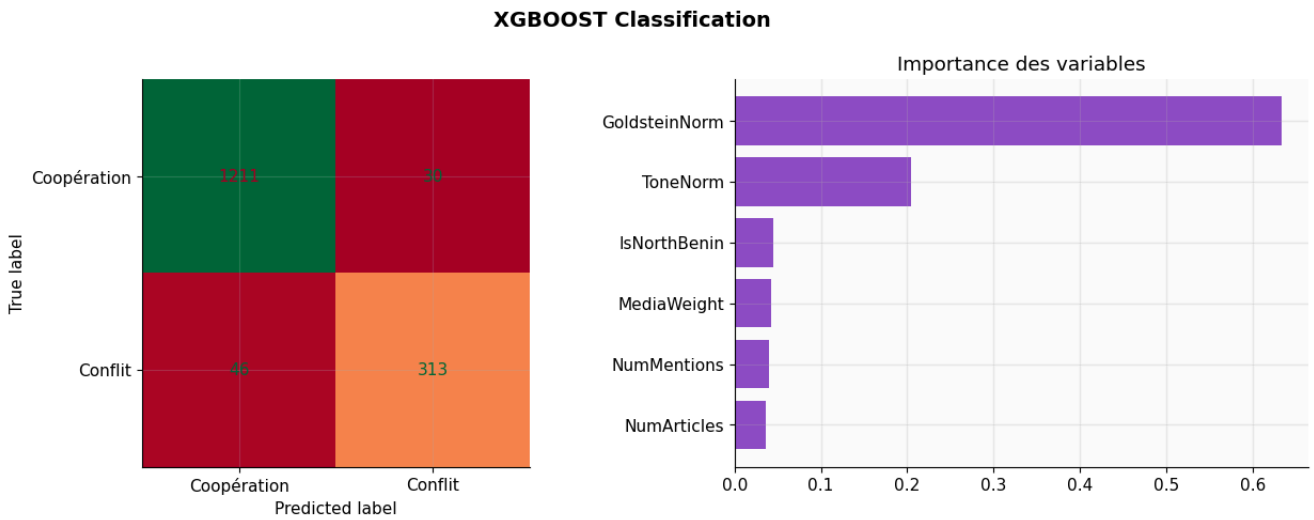
```
              precision    recall  f1-score   support

 Coopération       0.97      0.98      0.98      1241
     Conflit       0.94      0.89      0.91       359

    accuracy                           0.96      1600
   macro avg       0.95      0.93      0.94      1600
weighted avg       0.96      0.96      0.96      1600
```



XGBOOST

```
model_xgboost = XGBClassifier(use_label_encoder=False, eval_metric='logloss')
model_xgboost.fit(X_train, y_train)

y_pred = model_xgboost.predict(X_test)

# print(classification_report(y_test, y_pred))

report=classification_report(y_test,y_pred,target_names=['Coopération','Conflit'],output_dict=True)
report_models['xgboost'] = report
cm=confusion_matrix(y_test,y_pred)
print(classification_report(y_test,y_pred,target_names=['Coopération','Conflit']))

fig,axes=plt.subplots(1,2,figsize=(13,5))
fig.suptitle('XGBOOST Classification',fontsize=14,fontweight='bold')
ConfusionMatrixDisplay(cm,display_labels=['Coopération','Conflit']).plot(ax=axes[0],cmap='RdYlGn',colorbar=False)
imp=pd.Series(model_xgboost.feature_importances_,index=features).sort_values()
axes[1].barh(imp.index,imp.values,color=COLORS['secondary'],alpha=.85)
axes[1].set_title('Importance des variables')
plt.tight_layout()
plt.savefig(DATA_PROC/'viz_rf.png',dpi=150,bbox_inches='tight')
plt.show()
```

```
c:\Users\TSS\anaconda3\Lib\site-packages\xgboost\training.py:200: UserWarning: [10:20:10] WARNING: C:\actions-runner\_work\xgboost\xgbo
Parameters: { "use_label_encoder" } are not used.

  bst.update(dtrain, iteration=i, fobj=obj)
              precision    recall  f1-score   support

 Coopération       0.96      0.98      0.97      1241
     Conflit       0.91      0.87      0.89       359

    accuracy                           0.95      1600
   macro avg       0.94      0.92      0.93      1600
weighted avg       0.95      0.95      0.95      1600
```




```
df['proba_conflict'] = model_xgboost.predict_proba(X)[:,1]
df.to_csv('../data/processed/gdelt_models_data.csv', index = False)
```

```
# Sauvegarde modèles
pickle.dump({'scaler':scaler,'kmeans':kmeans,'pca':pca, 'dbscan':dbscan, 'perl':model_perl,'rf':model_rf,
            'xgboost':model_xgboost,'feat':features, 'best_k':best_k,'sil_scores':sil,'report':report,
            'profile':profile.to_dict()}, open(MODELS/'models_bundle.pkl','wb'))
print('models/models_bundle.pkl sauvegardé')
```

```
models/models_bundle.pkl sauvegardé
```